

Google SWE Interview Roadmap (C++ Specialization)

Objective: Convert "brushed up DSA" into "Google Hire" caliber performance. **Constraint:** Google's C++ standards are idiosyncratic. Standard "LeetCode C++" often fails the code quality bar at Google.

Phase 1: The "Google C++" Standard (Week 1)

Goal: Stop writing "Competitive Programming C++" and start writing "Production C++". *Crucial Reality:* If you write `using namespace std;` or manually manage memory with `new` and `delete`, you are signalling that you are a junior engineer or stuck in 1998.

1. The Google Style Guide (Non-Negotiable)

Google has a specific [Open Source Style Guide](#). You don't need to memorize it all, but you **must** adhere to these during the interview:

- **No Exceptions:** Google code is generally exception-free. Do not use `try`, `catch`, or `throw`.
 - *Instead:* Return status codes (bool, optional, or pointers).
- **Variable Naming:** `snake_case` for local variables and arguments.
- **Function Naming:** `PascalCase` (CapitalizedWords).
- **Member Variables:** `snake_case_` (trailing underscore is mandatory for private members).
- **Headers:** Always use `#pragma once` or standard include guards.

2. Modern C++ Mandates

- **Smart Pointers:** NEVER use `malloc` / `free`. Rarely use `new` / `delete`.

- Use `std::unique_ptr` by default (exclusive ownership).
- Use `std::shared_ptr` only when ownership is truly shared.
- **Casts:** No C-style casts (`(int)x`). Use `static_cast<int>(x)` .
- **Const Correctness:** If a method doesn't modify the class state, it **must** be marked `const` . If a variable shouldn't change, it **must** be `const` .

3. Google C++ Checklist

- ☐ I can explain why `std::vector` is preferred over `std::list` (cache locality).
- ☐ I know how to use `std::move` to avoid expensive copies.
- ☐ I can write a custom comparator for `std::priority_queue` (min-heap) without looking it up.
- ☐ I know the difference between `std::map` (Red-Black Tree, $O(\log N)$) and `std::unordered_map` (Hash Table, $O(1)$ avg).

Phase 2: Implementation Patterns (Week 2)

Goal: Speed and Standard Library Mastery. *Harsh Truth:* If you implement a Linked List from scratch when you could have used `std::list` (unless explicitly asked to), you failed the "Knowledge of Tools" competency.

1. Essential STL Mastery

- **Vectors:** `reserve()` vs `resize()` . Know how capacity doubling works.
- **Maps:** Using `emplace()` vs `insert()` . Handling missing keys in `unordered_map` without double-lookup (use `find()` or `try_emplace`).
- **Strings:** Use `std::string` methods (`find` , `substr`). Bonus: Mention `std::string_view` for read-only parameters to avoid allocations (C++17).

2. Algorithms Library (`<algorithm>`)

Don't reinvent the wheel. Use these:

- `std::sort` (IntroSort: Hybrid of QuickSort, HeapSort, InsertionSort).
- `std::lower_bound` / `std::upper_bound` (Binary Search).
- `std::next_permutation` (Great for "find all combinations" problems).
- `std::unique` (remove duplicates, but remember to `erase` after).

3. Code Snippet: The "Google Style" Setup

Practice writing your solutions in this format on a blank Google Doc (no IDE).

```
// Bad (Competitive Programming Style)
#include <bits/stdc++.h>
using namespace std;

// Good (Google Interview Style)
#include <vector>
#include <string>
#include <optional>
#include <algorithm>

// Use PascalCase for functions
std::optional<int> FindTargetValue(const std::vector<int>& data, int target) {
    // Use snake_case for variables
    for (size_t i = 0; i < data.size(); ++i) {
        if (data[i] == target) {
            return i;
        }
    }
    return std::nullopt; // Explicitly handle "not found"
```

```
}
```

Phase 3: System Design & Concurrency (Week 3)

Goal: Handle L4+ scale and complexity questions.

1. Concurrency in C++

You may be asked to make your code thread-safe.

- **Mutexes:** `std::mutex` , `std::lock_guard` (RAII locking), `std::unique_lock` .
- **Atomics:** `std::atomic<T>` for lock-free counters.
- **Condition Variables:** `std::condition_variable` for producer-consumer problems.
- **Pitfall:** Know what "Deadlock" is and how to prevent it (always acquire locks in the same order).

2. Large Scale Concepts

- **Memory constraints:** If input is 1TB and RAM is 16GB, how do you sort? (External Merge Sort).
- **MapReduce:** Understand the concept, even if you don't code it.

Phase 4: Execution Strategy (The Interview)

Goal: flawless execution.

1. The "Google Doc" Environment

- You will likely code in a Google Doc or a plain text editor.
- **No Syntax Highlighting.**

- **No Auto-completion.**
- *Drill:* Practice writing code in Notepad or a Google Doc. If you rely on red squiggly lines to find syntax errors, you will panic in the interview.

2. Communication Protocol

1. **Clarify:** "Are the inputs sorted?", "Can the array be empty?", "Is the graph cyclic?"
2. **Constraints:** "What is the max value of N? Does this need to fit in memory?"
3. **Brute Force First:** "I can solve this in $O(N^2)$ using nested loops, but we can optimize."
4. **Optimize:** "Using a Hash Map, we can bring this down to $O(N)$."
5. **Coding:** Write clean, Google-style C++.
6. **Dry Run: CRITICAL.** Manually step through your code with a sample input (write variable values in comments) *before* you tell the interviewer you are done.

3. Common C++ Traps to Avoid

- **Iterator Invalidation:** modifying a vector while iterating over it.
- **Slicing:** Passing a derived class by value to a function expecting a base class. (Always pass by reference/pointer).
- **Undefined Behavior:** Accessing `vector[i]` without checking bounds. Use `vector.at(i)` if you want safety, or manual checks.

Mock Interview Checklist

Concept	Proficiency	Notes
---------	-------------	-------

Pointers/Refs	High	<code>*</code> vs <code>&</code> , passing by <code>const</code> &
Memory	High	Stack vs Heap, <code>unique_ptr</code>
Graph Algo	High	BFS/DFS (Iterative implementation pref.)
Trees	High	Traversal (In/Pre/Post), Tries
Heaps	High	<code>priority_queue</code> syntax & custom comparator
Bit Manip	Med	XOR tricks, masking
String Parsing	High	<code>stringstream</code> usage

Final "Harsh" Advice

Google interviewers look for "signals."

- **Positive Signal:** You use `const` correctly everywhere. You validate inputs immediately. You ask about integer overflow.
- **Negative Signal:** You use global variables. You use `int` for array sizes (use `size_t`). You stay silent for more than 60 seconds while thinking.